PROJET PRATIQUE : Créer la Boutique en Ligne de votre Jeu de Cartes

Objectif du projet : Vous avez conçu la base de données de votre jeu vidéo sur phpMyAdmin. Nous allons maintenant lui donner vie. Vous allez créer un site web où un joueur pourra utiliser ses points virtuels pour acheter des cartes qui s'ajouteront automatiquement à son inventaire dans votre base SQL.

PARTIE 1 : Comprendre l'Architecture "3 Tiers"

Avant d'écrire la moindre ligne de code, il faut comprendre comment fonctionne un site web moderne. Une règle de sécurité absolue en informatique dicte que **le code de votre page web ne doit JAMAIS communiquer directement avec votre base de données SQL**. Si vous mettez le mot de passe de votre base de données dans le code de la page web, n'importe quel joueur pourrait faire un clic droit > "Inspecter" et pirater tout le jeu.

Pour sécuriser cela, nous utilisons une architecture en **3 Tiers** (3 niveaux). Imaginez un restaurant :

1. **Le Frontend (La salle de restaurant) :** C'est votre page web (HTML, CSS et JavaScript de base). C'est la vitrine de la boutique que le client voit. Le client passe commande, mais il n'a pas le droit d'entrer en cuisine.
2. **Le Backend (Le serveur en salle) :** C'est un programme qui tourne sur la machine hôte. Il reçoit la commande du client, vérifie s'il a assez d'argent, et transmet la demande à la cuisine.
3. **La Base de données (La cuisine) :** C'est votre serveur MySQL (phpMyAdmin). Il stocke les ingrédients (les cartes, les joueurs, les inventaires). Il ne parle qu'au serveur Backend, jamais au client final.

PARTIE 2 : Installation des Outils Serveur (Node.js et npm)

Pour créer notre Backend (le serveur), nous allons utiliser le langage JavaScript. Pour que JavaScript puisse fonctionner sur votre ordinateur (et pas

seulement dans un navigateur web), vous avez besoin d'un outil appelé **Node.js**. Node.js est livré avec un outil indispensable : **npm** (Node Package Manager), qui est en quelque sorte "l'App Store" des développeurs.

Étape 2.1 : Installer Node.js sur votre poste

1. Ouvrez votre navigateur et allez sur **nodejs.org**.
2. Téléchargez la version recommandée pour la plupart des utilisateurs (version LTS).
3. Lancez l'installateur et cliquez sur "Suivant" en laissant toutes les options par défaut.
4. Une fois l'installation terminée, **redémarrez Visual Studio Code** s'il était ouvert.

Étape 2.2 : Vérifier l'installation

1. Ouvrez Visual Studio Code.
2. Ouvrez un terminal intégré en cliquant sur le menu `Terminal > Nouveau Terminal`.
3. Tapez `node -v` et appuyez sur Entrée. Un numéro de version (ex: v20.x.x) doit s'afficher.
4. Tapez `npm -v` et appuyez sur Entrée. Un autre numéro doit s'afficher. Si c'est le cas, votre poste est prêt.

PARTIE 3 : Création et Structure du Projet

Nous allons maintenant préparer l'arborescence de notre site web.

Étape 3.1 : Initialiser le dossier

1. Créez un nouveau dossier sur votre bureau nommé `boutique_jeu`.
2. Dans Visual Studio Code, faites `Fichier > Ouvrir le dossier` et sélectionnez `boutique_jeu`.
3. Dans le terminal de VS Code, tapez la commande suivante pour initialiser le projet :

```
npm init -y
```

Cette commande génère un fichier `package.json`. C'est la carte d'identité de votre projet. Il listera toutes les bibliothèques externes dont votre serveur aura besoin pour fonctionner.

Étape 3.2 : Installer les modules du serveur
Dans le terminal, tapez :

```
npm install express mysql2 cors
```

Voici ce que vous venez de télécharger grâce à npm :

- **express :** Un outil qui permet de créer un serveur web facilement.
- **mysql2 :** Le traducteur qui permettra à votre serveur de parler avec phpMyAdmin.
- **cors :** Une sécurité qui autorisera votre page web HTML à discuter avec votre serveur Backend.

Étape 3.3 : Comprendre l'arborescence finale
Créez maintenant trois fichiers vides dans VS Code : `server.js`, `index.html` et `script.js`.
Regardez l'explorateur de fichiers à gauche dans VS Code. Votre projet doit exactement ressembler à ceci :

- `node_modules/` : Un gros dossier généré automatiquement par npm. Il contient tout le code de `express`, `mysql2` et `cors`. **Règle d'or : on n'ouvre jamais et on ne modifie jamais ce dossier.**
- `package-lock.json` : Un fichier généré automatiquement qui verrouille les versions exactes de vos outils. N'y touchez pas.
- `package.json` : La carte d'identité de votre projet (créée à l'étape 3.1).
- `server.js` : Votre Backend. C'est ici que nous allons coder la logique du jeu et la connexion SQL.
- `index.html` : Votre Frontend. C'est le design de la boutique.
- `script.js` : Le JavaScript de votre Frontend. C'est lui qui détectera les clics sur les boutons "Acheter" et enverra la demande au `server.js`.

PARTIE 4 : Connecter le Serveur à la Base de Données

Ouvrez votre fichier `server.js`. Nous allons écrire le code qui lance le serveur et le connecte à votre base MySQL.

```
// Importation des bibliothèques téléchargées via npm
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');

const app = express();
app.use(cors()); // Autorise le Frontend à parler au Backend
app.use(express.json()); // Permet au serveur de lire les données envoyées par le Fron
tend

// 1. CONNEXION À LA BASE DE DONNÉES
const db = mysql.createConnection({
    host: 'localhost',
    user: 'root',      // Nom d'utilisateur par défaut de phpMyAdmin
    password: '',      // Mot de passe par défaut (vide sous XAMPP/WAMP)
    database: 'e_gaming' // ATTENTION : Mettez ici le nom exact de VOTRE base de donné
es !
});

db.connect((err) => {
    if (err) {
        console.error('Erreur de connexion à la BDD :', err);
    } else {
        console.log('✅ Connecté avec succès à la base de données MySQL !');
    }
});

// Lancement du serveur sur le port 3000
app.listen(3000, () => {
    console.log('🚀 Serveur en ligne sur http://localhost:3000');
});
```

Allumez WAMP ou XAMPP pour que votre base de données soit active. Dans le terminal de VS Code, tapez `node server.js`. Si le message ✅ s'affiche, le pont entre votre serveur Node et votre base SQL est opérationnel !

PARTIE 5 : Créer la "Route" d'Achat (La logique métier)

Toujours dans `server.js`, nous allons créer une "route" (une URL invisible) que le Frontend appellera quand le joueur cliquera sur "Acheter".
Ajoutez ce code juste avant la ligne `app.listen` :

```
// 2. ROUTE POUR ACHETER UNE CARTE
app.post('/acheter-carte', (req, res) => {
```

```
    // On récupère les informations envoyées par la page HTML/JS
    const id_joueur = req.body.id_joueur;
    const id_carte = req.body.id_carte;
    const prix_carte = req.body.prix;

    // Étape A : Vérifier si le joueur possède assez de points
    const requeteVerif = 'SELECT points FROM joueur WHERE id_joueur = ?';

    db.query(requeteVerif, [id_joueur], (err, resultats) => {
        if (err) return res.status(500).send("Erreur de base de données");

        const pointsDuJoueur = resultats[0].points;

        if (pointsDuJoueur >= prix_carte) {
            // Étape B : Le joueur a assez d'argent. On déduit les points.
            const nouveauxPoints = pointsDuJoueur - prix_carte;
            const requetePaiement = 'UPDATE joueur SET points = ? WHERE id_joueur =
?';

            db.query(requetePaiement, [nouveauxPoints, id_joueur], (err, resultatPaiem
ent) => {

                // Étape C : On ajoute la carte dans l'inventaire du joueur
                const requeteInventaire = 'INSERT INTO inventaire (id_joueur, id_cart
e) VALUES (?, ?)';

                db.query(requeteInventaire, [id_joueur, id_carte], (err, resultatInven
taire) => {
                    res.json({
                        succes: true,
                        message: "🎉 Carte ajoutée à votre inventaire !",
                        points_restants: nouveauxPoints
                    });
                });
            });

        } else {
            // Le joueur n'a pas assez de points
            res.json({
                succes: false,
                message: "❌ Fonds insuffisants pour acheter cette carte."
            });
        }
    });
});
```

Ce code vérifie les points (`SELECT`), encaisse le paiement (`UPDATE`) et livre la marchandise dans la table d'association (`INSERT INTO`).

PARTIE 6 : Le Frontend (L'interface Utilisateur)

Ouvrez le fichier `index.html`. C'est ici que l'on construit ce que le joueur va voir sur son écran.

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Boutique E-Gaming</title>
    <style>
        body { font-family: Arial; background-color: #1e1e2f; color: white; text-alig
n: center; }
        .carte { border: 2px solid #ffd700; padding: 20px; display: inline-block; marg
in: 20px; border-radius: 10px; background-color: #2a2a40; }
        button { background-color: #ffd700; border: none; padding: 10px 20px; cursor:
pointer; font-weight: bold; border-radius: 5px;}
        button:hover { background-color: #ffaa00; }
        #points-display { font-size: 24px; color: #00ff00; font-weight: bold;}</style>
</head>
<body>

    <h1>Bienvenue dans la Boutique</h1>
    <p>Vos points : <span id="points-display">150</span> 💰</p>

    <div class="carte">
        <h2>Dragon Blanc</h2>
        <p>Attaque : 120 | PV : 200</p>
        <p>Prix : 50 points</p>
        <!-- Appel de la fonction JS avec : ID du joueur, ID de la carte, Prix -->
        <button onclick="acheterCarte(1, 2, 50)">Acheter (50 pts)</button>
    </div>

    <div class="carte">
        <h2>Le Gobelin</h2>
        <p>Attaque : 30 | PV : 50</p>
        <p>Prix : 10 points</p>
        <button onclick="acheterCarte(1, 4, 10)">Acheter (10 pts)</button>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

PARTIE 7 : Relier le Frontend au Backend

Ouvrez le fichier `script.js`. Nous allons utiliser la fonction `fetch()` pour passer la commande de l'utilisateur depuis son navigateur web vers le serveur Node.js.

```
function acheterCarte(idJoueur, idCarte, prix){
```

```
    // 1. Préparation du "colis" contenant les infos de l'achat
    const donneesTransaction = {
        id_joueur: idJoueur,
        id_carte: idCarte,
        prix: prix
    };

    // 2. Envoi du colis au Backend via fetch()
    fetch('http://localhost:3000/acheter-carte', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(donneesTransaction)
    })
    .then(reponse => reponse.json()) // On lit la réponse du serveur
    .then(data => {

        // 3. Traitement du résultat
        if (data.succes === true) {
            alert(data.message);
            // Mise à jour visuelle immédiate des points du joueur
            document.getElementById('points-display').innerText = data.points_restant
s;
        } else {
            alert(data.message); // Affiche l'erreur (ex: pas assez de points)
        }
    })
    .catch(erreur => {
        console.error("Erreur technique :", erreur);
        alert("Impossible de contacter le serveur de la boutique.");
    });
}
```

PARTIE 8 : Les Défis

Votre boutique de base est opérationnelle. Pour valider vos compétences avancées, choisissez et réalisez l'un des défis suivants :

1. **Le défi de l'Affichage Dynamique :** Actuellement, le HTML écrit "150 points" en dur. Créez une nouvelle route Backend (`app.get('/mes-points/:id')`) qui va chercher le solde réel du joueur dans la base de données au chargement de la page, et utilisez JavaScript pour afficher ce vrai montant.
2. **Le défi Anti-Triche :** Que se passe-t-il si un joueur possède déjà la carte "Dragon Blanc" et clique de nouveau sur Acheter ? Modifiez la requête SQL de la Partie 5 pour interroger la table `inventaire`. Si la carte s'y trouve déjà pour ce joueur, bloquez l'achat et renvoyez un message d'erreur.

3. **Le défi Cosmétique :** Ajoutez une nouvelle table `skins` à votre base de données. Créez les boutons HTML et une nouvelle route `/acheter-skin` dans le `server.js` pour permettre l'achat de visuels alternatifs pour les decks.